AI Global Architecture

Comprehensive Diagrams & Component Specifications

Agent Architecture | Communication Protocols | AI Components

Senior AI Engineer Architect Perspective

Design Patterns & Best Practices

Document Version: 1.0

Table of Contents

1. Agent Orchestration Architecture

2. Agent Communication Protocols

3. Individual Agent Specifications

4. NLP Architecture

5. Computer Vision Architecture

6. Voice Processing Architecture

7. TTS & Avatar Architecture

8. RAG & Knowledge Architecture

9. AI Design Patterns

10. Model Serving Architecture

1. Agent Orchestration Architecture

The Agent Orchestration Architecture defines the coordination layer that manages multiple AI agents working together to conduct comprehensive candidate interviews. This architecture employs the ReAct (Reasoning and Acting) pattern combined with multi-agent collaboration strategies to ensure coherent, context-aware interview experiences. The orchestration layer handles state management, task routing, inter-agent communication, and fallback mechanisms for robust operation.

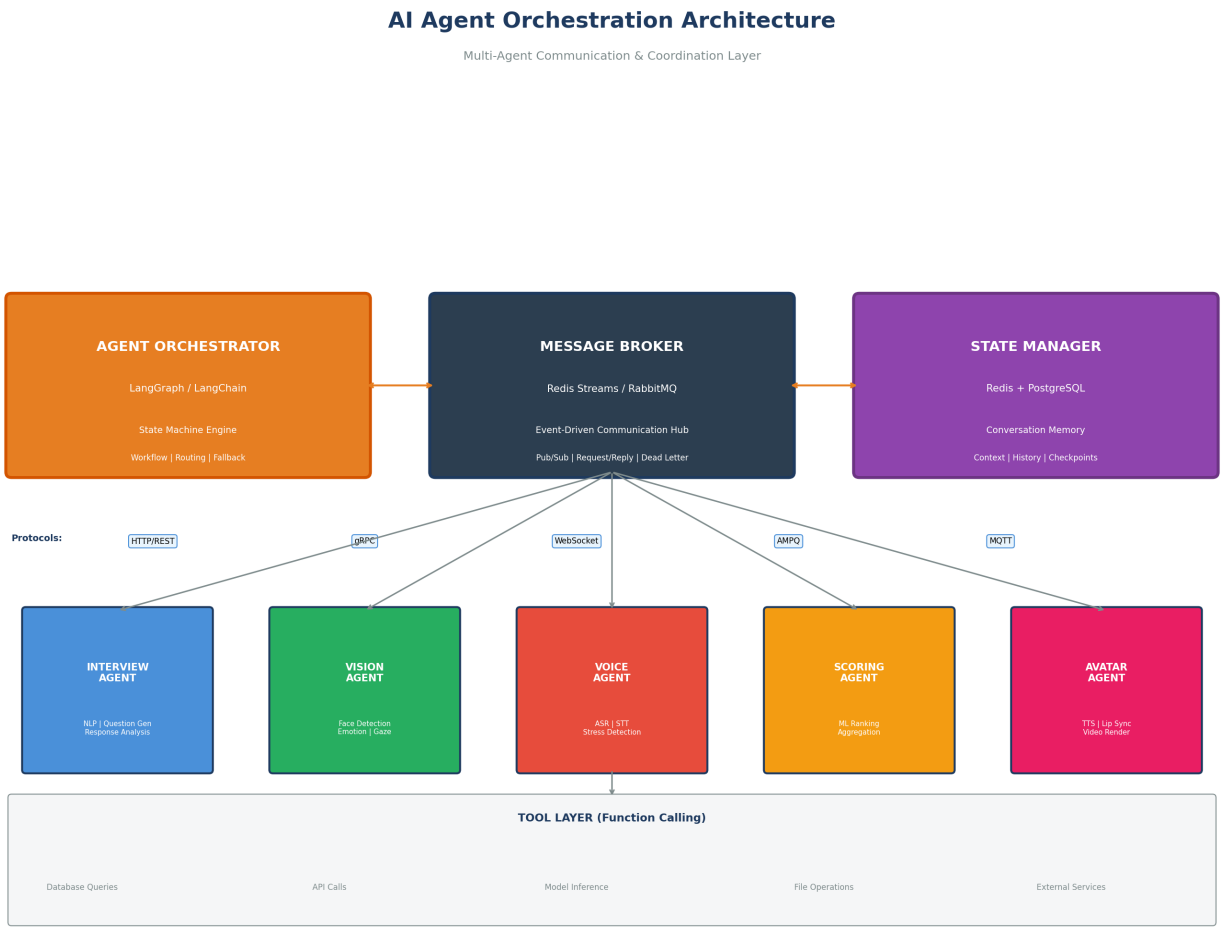

Figure 1: Agent Orchestration Architecture

1.1 Core Components

The orchestration system comprises three primary coordination components that work in concert to manage the distributed agent ecosystem. The Agent Orchestrator implements a state machine engine based on LangGraph, providing deterministic workflow execution with support for branching logic, error recovery, and checkpoint-based state persistence. The Message Broker serves as the central communication hub, implementing both publish-subscribe and request-reply patterns through Redis Streams or RabbitMQ. The State Manager maintains conversation history, candidate context, and interview progress using a combination of Redis for fast access and PostgreSQL for durable storage.

1.2 Component Definitions

Component	Technology	Purpose	Pattern
Agent Orchestrator	LangGraph / LangChain	Workflow coordination, task routing, state transitions	State Machine
Message Broker	Redis Streams / RabbitMQ	Async communication, event distribution, queuing	Pub/Sub
State Manager	Redis + PostgreSQL	Conversation memory, context persistence, checkpoints	Repository
Tool Layer	Function Calling API	External integrations, database access, API calls	Adapter

Table 1: Orchestration Component Definitions

2. Agent Communication Protocols

The communication protocols define how agents exchange information, coordinate tasks, and maintain consistency across the distributed system. The architecture implements a layered protocol stack that supports both synchronous request-response and asynchronous event-driven communication patterns. This flexibility enables the system to handle real-time interview interactions while maintaining scalability and fault tolerance.

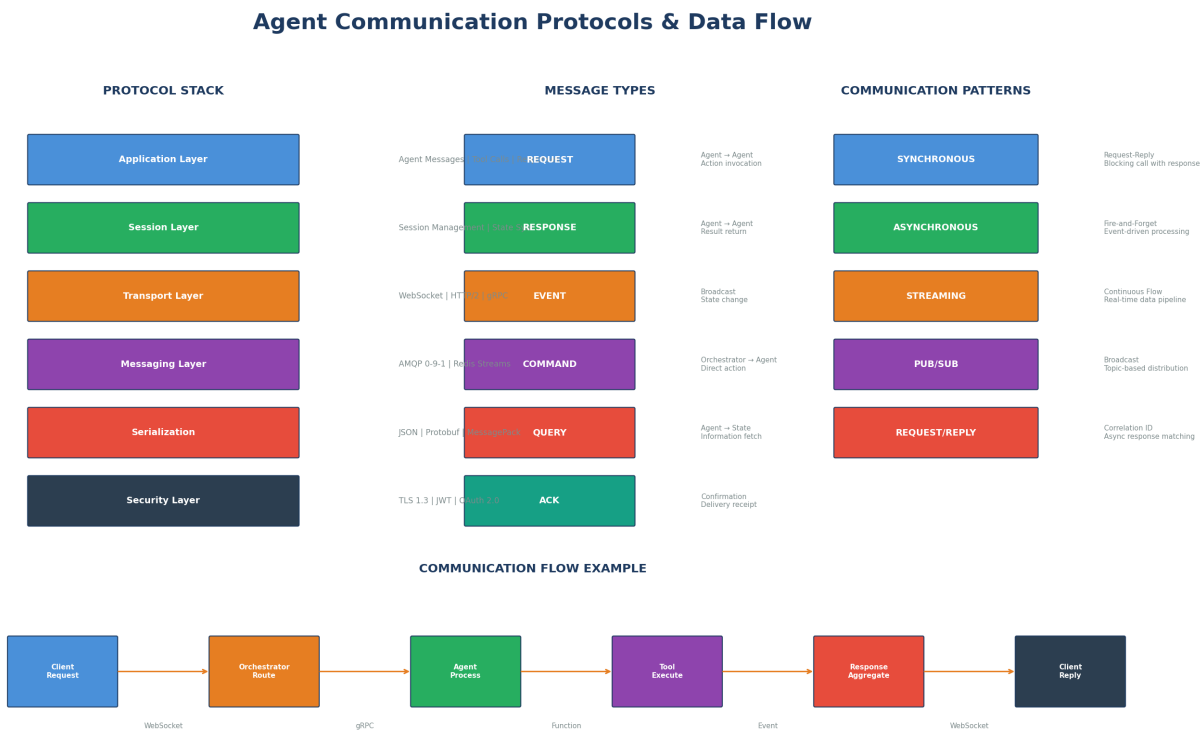

Figure 2: Agent Communication Protocols & Data Flow

2.1 Protocol Definitions

HTTP/REST: Synchronous request-response protocol for standard API calls. Used for candidate registration, interview initialization, and report retrieval. Implements idempotency keys for safe retries.

WebSocket: Full-duplex communication channel for real-time bidirectional data exchange. Primary protocol for live interview sessions, enabling instant message delivery and video stream coordination.

gRPC: High-performance RPC framework using Protocol Buffers for efficient binary serialization. Employed for inter-service communication between AI agents, reducing latency in high-throughput scenarios.

AMQP 0-9-1: Advanced Message Queuing Protocol for reliable asynchronous messaging. Implements message acknowledgment, dead-letter queues, and guaranteed delivery for critical interview events.

Redis Streams: Log-based data structure for event sourcing and message streaming. Used for real-time event distribution, audit logging, and implementing the event sourcing pattern.

2.2 Message Types

Type	Direction	Purpose	Example
REQUEST	Agent → Agent	Action invocation with parameters	analyze_response(text, context)
RESPONSE	Agent → Agent	Result return from action	score: 0.85, sentiment: positive
EVENT	Broadcast	State change notification	interview_phase_changed
COMMAND	Orchestrator → Agent	Direct action instruction	start_evaluation(candidate_id)
QUERY	Agent → State Manager	Information retrieval	get_interview_context(session_id)
ACK	Receiver → Sender	Delivery confirmation	message_id: 12345, status: received

Table 2: Message Type Definitions

3. Individual Agent Specifications

Each agent in the system is designed as an autonomous, specialized component with clearly defined responsibilities, tools, and action capabilities. The agent architecture follows the principle of single responsibility while enabling rich inter-agent collaboration through standardized interfaces. Below are detailed specifications for each agent type.

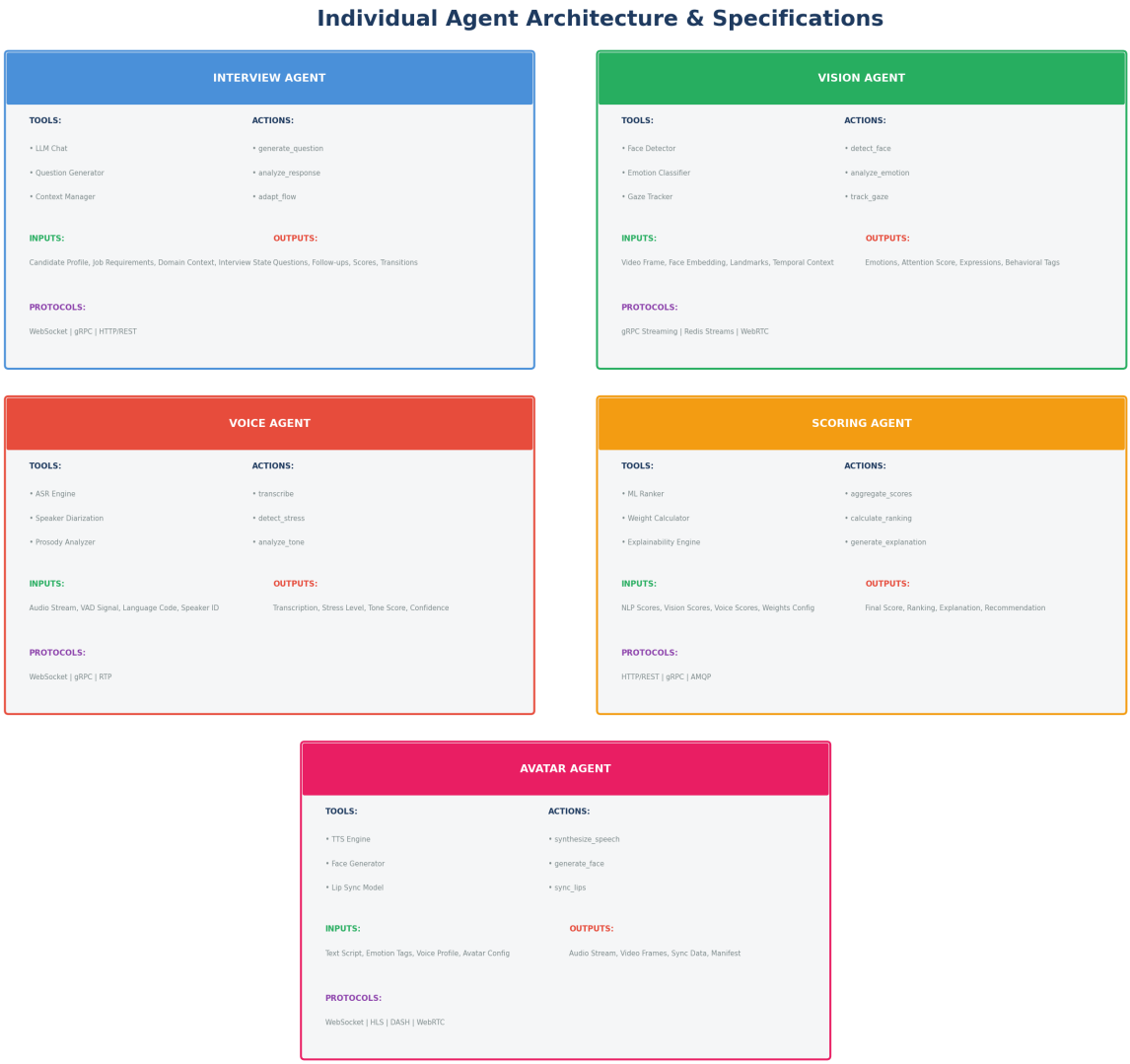

Figure 3: Individual Agent Architecture & Specifications

3.1 Interview Agent

The Interview Agent serves as the primary conversational AI responsible for conducting the interview dialogue with candidates. It leverages Large Language Models (LLMs) fine-tuned for interview contexts to generate contextually relevant questions, analyze candidate responses, and adapt the interview flow based on real-time assessment of answer quality.

Category	Components

Tools	LLM Chat Engine, Question Generator, Context Manager, RAG Retrieval, Response Analyzer, Quiz Generator
Actions	generate_question(domain, difficulty), analyze_response(text, context), adapt_flow(score, phase), score_answer(response, criteria), generate_followup(topic, depth)
Inputs	Candidate Profile, Job Requirements, Domain Context, Interview State, Previous Responses, Time Constraints
Outputs	Questions, Follow-up Queries, Response Scores, Phase Transitions, Interview Progress, Completion Status
Protocols	WebSocket (real-time chat), gRPC (internal calls), HTTP/REST (configuration)
LLM Backends	GPT-4 Turbo (primary), Claude 3 Opus (fallback), Llama 3 (on-premise option)

Table 3: Interview Agent Specification

3.2 Vision Agent

The Vision Agent processes video streams from the interview session to extract behavioral signals and emotional indicators. It employs state-of-the-art computer vision models for face detection, emotion recognition, gaze tracking, and posture analysis. The agent outputs behavioral metrics that contribute to the holistic candidate evaluation.

Category	Components
Tools	Face Detector (MTCNN/RetinaFace), Emotion Classifier (FER Model), Gaze Tracker (Eye Aspect Ratio), Pose Estimator (MediaPipe), Attention Analyzer
Actions	detect_face(frame), analyze_emotion(face_crop), track_gaze(landmarks), assess_posture(pose_keypoints), compute_attention_score(sequence)
Inputs	Video Frame (RGB), Face Embeddings, Facial Landmarks, Temporal Context, Calibration Data
Outputs	Emotion Distribution, Attention Score (0-100), Behavioral Tags, Expression Variance, Posture Quality, Engagement Level
Protocols	gRPC Streaming (video frames), Redis Streams (metrics), WebRTC (source)
Models	RetinaFace (detection), EfficientNet-B2 (emotion), MediaPipe Face Mesh (gaze), BlazePose (posture)

Table 4: Vision Agent Specification

3.3 Voice Agent

The Voice Agent handles all audio processing including automatic speech recognition, speaker diarization, prosodic analysis, and stress detection. It transforms raw audio streams into structured transcripts enriched with acoustic features that indicate confidence, stress, and emotional state.

Category	Components
Tools	ASR Engine (Whisper), VAD (Silero), Speaker Diarization (Pyannote), Prosody Analyzer, Stress Detection Model, Sentiment Analyzer
Actions	transcribe(audio_chunk), detect_stress(prosody_features), analyze_tone(spectral_data), measure_confidence(hesitations, fillers), detect_emotion(audio)
Inputs	Audio Stream (16kHz PCM), VAD Signal, Language Code, Speaker ID, Audio Calibration
Outputs	Transcription (text), Word Timestamps, Stress Level (0-100), Tone Score, Confidence Metric, Emotion Distribution
Protocols	WebSocket (streaming audio), gRPC (transcription service), RTP (real-time transport)
Models	Whisper Large-v3 (ASR), Wav2Vec2 (SER), Silero VAD, Pyannote Audio (diarization)

Table 5: Voice Agent Specification

3.4 Scoring Agent

The Scoring Agent serves as the final evaluation layer, aggregating signals from all other agents into a comprehensive candidate assessment. It implements weighted scoring algorithms, machine learning ranking models, and explainability mechanisms to produce transparent, actionable evaluation results.

Category	Components
Tools	ML Ranker (XGBoost), Weight Calculator, Explainability Engine (SHAP), Report Generator, Recommendation Engine
Actions	aggregate_scores(nlp, vision, voice), calculate_ranking(features), generate_explanation(scores), create_report(session_id), recommend(scores, threshold)
Inputs	NLP Scores, Vision Scores, Voice Scores, Technical Quiz Results, Weight Configuration, Historical Baselines
Outputs	Final Score (0-100), Ranking Percentile, Explanation Report, Recommendation (hire/no-hire/review), Confidence Interval
Protocols	HTTP/REST (API calls), gRPC (internal aggregation), AMQP (event-driven updates)
Models	XGBoost Classifier (ranking), Random Forest (ensemble), SHAP Explainer, Calibration Model

Table 6: Scoring Agent Specification

3.5 Avatar Agent

The Avatar Agent generates a realistic AI interviewer with synchronized speech, facial expressions, and lip movements. It combines text-to-speech synthesis with neural face generation and lip-sync models to create an

engaging, human-like interview experience.

Category	Components
Tools	TTS Engine (ElevenLabs/Azure), Face Generator (StyleGAN), Lip Sync Model (Wav2Lip/SadTalker), Video Composer, Expression Controller
Actions	synthesize_speech(text, voice_profile), generate_face(emotion), sync_lips(audio, face), render_video(frames, audio), stream_output(video)
Inputs	Text Script, Emotion Tags, Voice Profile ID, Avatar Configuration, Timing Data
Outputs	Audio Stream (WAV/MP3), Video Frames (H.264), Sync Metadata, Streaming Manifest (HLS/DASH)
Protocols	WebSocket (real-time streaming), HLS/DASH (video delivery), WebRTC (live streaming)
Models	ElevenLabs Multilingual v2 (TTS), SadTalker (lip sync), StyleGAN2-ADA (face), First Order Motion Model

Table 7: Avatar Agent Specification

4. NLP Architecture

The Natural Language Processing architecture encompasses all text understanding and generation capabilities of the system. It implements a multi-stage pipeline from raw text input through embedding generation, model inference, and output post-processing. The architecture supports both transformer-based models and traditional NLP techniques for comprehensive language analysis.

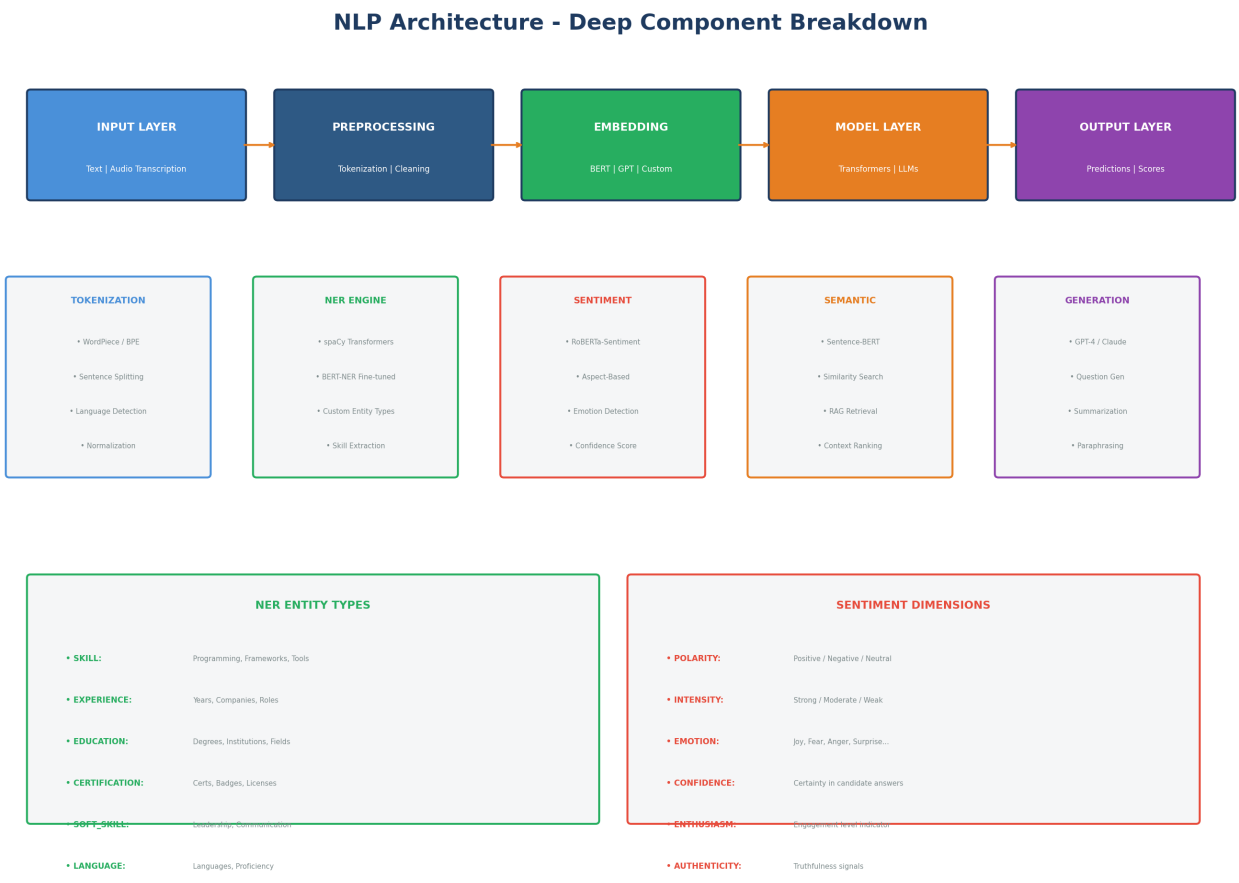

Figure 4: NLP Architecture - Deep Component Breakdown

4.1 NLP Component Definitions

Tokenization: The process of breaking text into tokens (words, subwords, or characters). Implements WordPiece and Byte Pair Encoding (BPE) algorithms used by modern transformer models. Includes sentence boundary detection and language identification.

Named Entity Recognition (NER): Identifies and classifies named entities in text such as skills, organizations, locations, dates, and custom entities relevant to recruitment. Uses transformer-based models fine-tuned on domain-specific data for extracting candidate qualifications, experience, and competencies.

Sentiment Analysis: Determines the emotional tone and opinion expressed in candidate responses. Implements aspect-based sentiment analysis to evaluate specific aspects of answers, emotion detection for multi-dimensional emotional classification, and confidence scoring for response certainty.

Semantic Similarity: Computes similarity between text segments using dense vector representations. Enables comparison of candidate responses against reference answers, RAG-based context retrieval, and semantic search across the knowledge base.

Text Generation: Produces natural language text using Large Language Models. Implements question generation for interview questions, summarization for candidate profiles, and paraphrasing for response variety.

5. Computer Vision Architecture

The Computer Vision architecture processes video streams to extract behavioral and emotional signals from candidates during interviews. It implements a real-time pipeline optimized for low-latency processing while maintaining high accuracy in face detection, emotion recognition, and behavioral analysis.

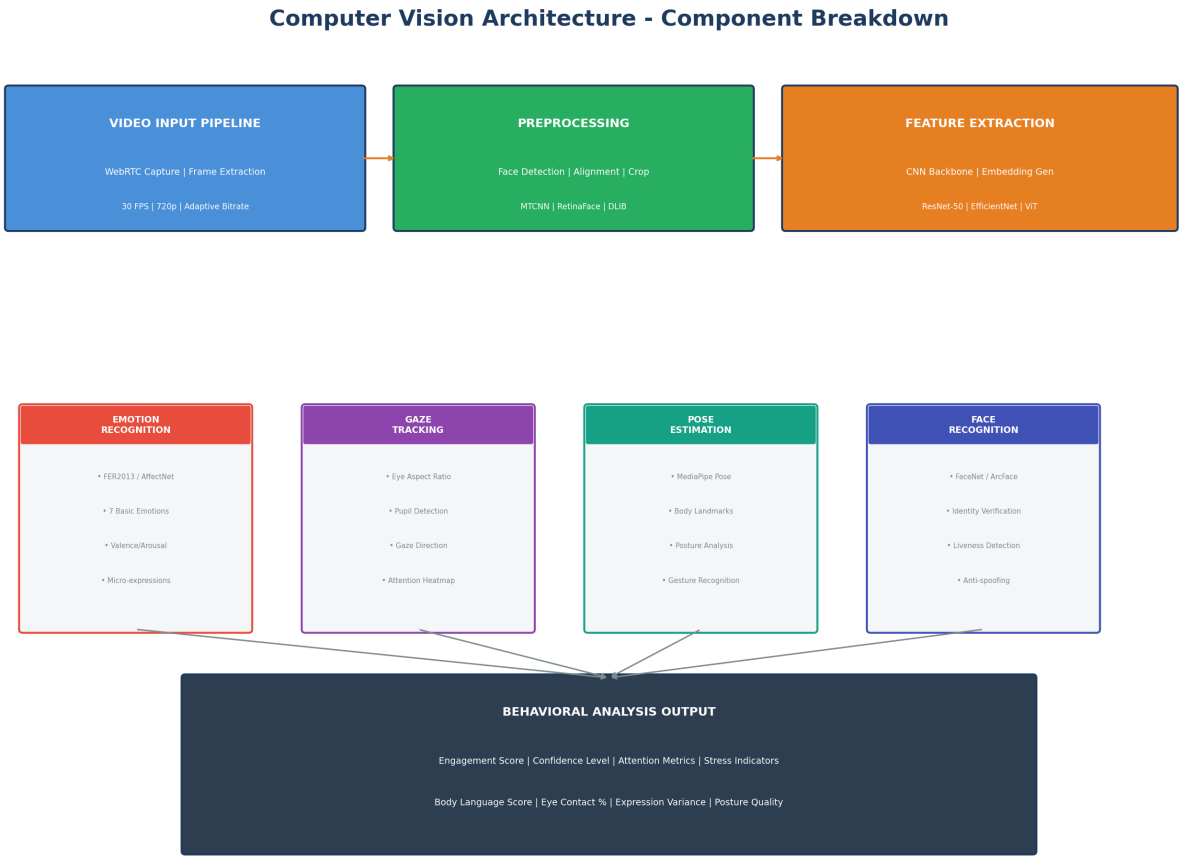

Figure 5: Computer Vision Architecture

5.1 Vision Component Definitions

Face Detection: Locates and extracts faces from video frames in real-time. Implements cascade classifiers for initial detection and deep learning models (MTCNN, RetinaFace) for precise localization. Handles multiple faces, partial occlusion, and varying lighting conditions.

Emotion Recognition: Classifies facial expressions into emotional states using convolutional neural networks trained on facial expression datasets. Outputs probability distributions over emotion categories (happy, sad, angry, fearful, surprised, disgusted, neutral) plus valence-arousal dimensions.

Gaze Tracking: Estimates eye gaze direction to measure attention and engagement. Computes Eye Aspect Ratio (EAR) for blink detection, pupil center localization, and gaze vector estimation. Generates attention heatmaps over time.

Pose Estimation: Detects body keypoints to analyze posture and body language. Uses MediaPipe or OpenPose for real-time skeleton tracking. Outputs posture quality scores, gesture recognition, and body movement patterns.

Face Recognition: Verifies candidate identity using face embeddings. Implements ArcFace or FaceNet for generating discriminative face representations. Includes liveness detection to prevent spoofing attacks.

6. Voice Processing Architecture

The Voice Processing architecture handles all audio-related processing including speech recognition, speaker analysis, and acoustic feature extraction. It transforms raw audio into structured, analyzable data enriched with prosodic and emotional information.

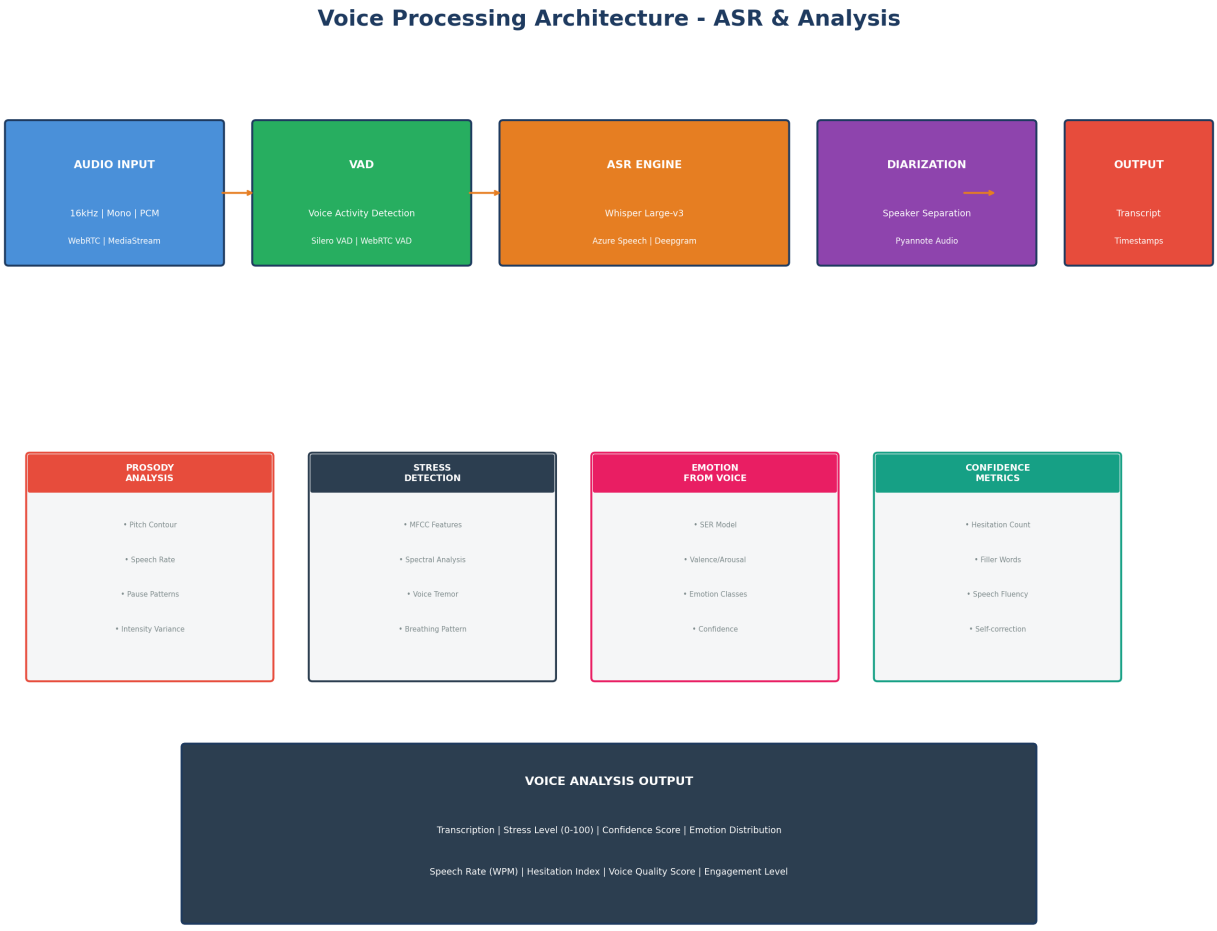

Figure 6: Voice Processing Architecture

6.1 Voice Component Definitions

Voice Activity Detection (VAD): Detects presence or absence of human speech in audio streams. Essential for segmenting audio into speech and non-speech regions. Implements Silero VAD and WebRTC VAD for robust detection across different acoustic environments.

Automatic Speech Recognition (ASR): Converts spoken audio into written text. Uses Whisper Large-v3 for multilingual transcription with word-level timestamps. Supports real-time streaming transcription and handles accents, background noise, and domain-specific terminology.

Speaker Diarization: Segments audio by speaker identity, answering 'who spoke when'. Essential for distinguishing between interviewer and candidate in two-way conversations. Uses Pyannote Audio for neural speaker embedding and clustering.

Prosody Analysis: Extracts acoustic features related to rhythm, stress, and intonation. Analyzes pitch contour, speech rate, pause patterns, and intensity variations. These features indicate confidence, enthusiasm, and stress

levels.

Speech Emotion Recognition (SER): Detects emotional states from acoustic properties of speech. Uses Wav2Vec2-based models fine-tuned on emotion datasets. Outputs emotion probabilities and dimensional emotion scores (valence, arousal, dominance).

7. TTS & Avatar Architecture

The Text-to-Speech and Avatar architecture creates a realistic AI interviewer capable of natural spoken interaction with synchronized facial animation. It combines neural speech synthesis with computer vision techniques to generate human-like video output in real-time.

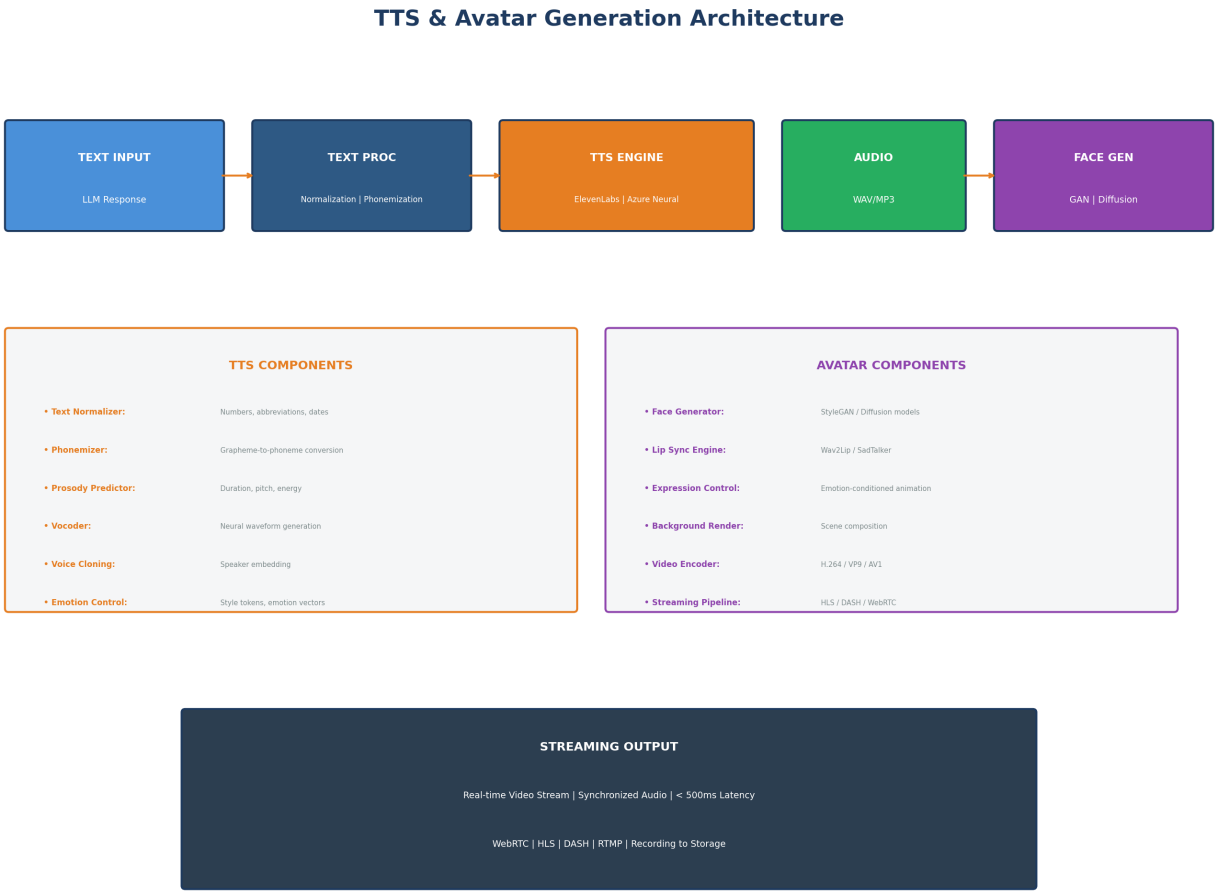

Figure 7: TTS & Avatar Generation Architecture

7.1 TTS Component Definitions

Text Normalization: Preprocesses input text for consistent pronunciation handling. Expands abbreviations, converts numbers to words, handles dates and currencies, and applies language-specific normalization rules.

Phonemization: Converts graphemes (written characters) to phonemes (speech sounds). Uses eSpeak NG or neural G2P models for accurate pronunciation across multiple languages and handling of proper names.

Prosody Prediction: Predicts duration, pitch contour, and energy for each phoneme. Uses neural networks trained on annotated speech data to generate natural-sounding prosody that conveys appropriate emotion and emphasis.

Neural Vocoder: Generates waveform from mel-spectrograms or other intermediate representations. Implements HiFi-GAN, BigVGAN, or WaveGlow for high-fidelity audio synthesis with minimal artifacts.

Voice Cloning: Creates custom voice profiles from audio samples. Uses speaker encoder networks to extract voice embeddings that can be applied to the TTS system for personalized avatar voices.

7.2 Avatar Component Definitions

Face Generation: Creates synthetic face images using generative models. Uses StyleGAN2-ADA or diffusion models to generate diverse, realistic faces. Supports conditioning on attributes like age, gender, and ethnicity.

Lip Synchronization: Animates face images to match audio speech. Implements Wav2Lip or SadTalker for audio-driven facial animation. Generates frame-by-frame lip movements synchronized with phonemes.

Expression Control: Modulates facial expressions based on emotional context. Uses emotion-conditioned animation models to generate appropriate expressions during speech. Supports eyebrow movement, head pose, and eye blinks.

Video Composition: Composites animated face with background and other visual elements. Handles video encoding (H.264, VP9, AV1) and packaging for streaming protocols.

Streaming Pipeline: Manages real-time video delivery to clients. Implements HLS, DASH, or WebRTC for adaptive streaming. Handles buffering, quality adaptation, and synchronization with audio.

8. RAG & Knowledge Architecture

The Retrieval-Augmented Generation (RAG) architecture provides the knowledge foundation for intelligent interview question generation and response evaluation. It combines dense vector retrieval with generative models to produce contextually relevant, factually grounded content.

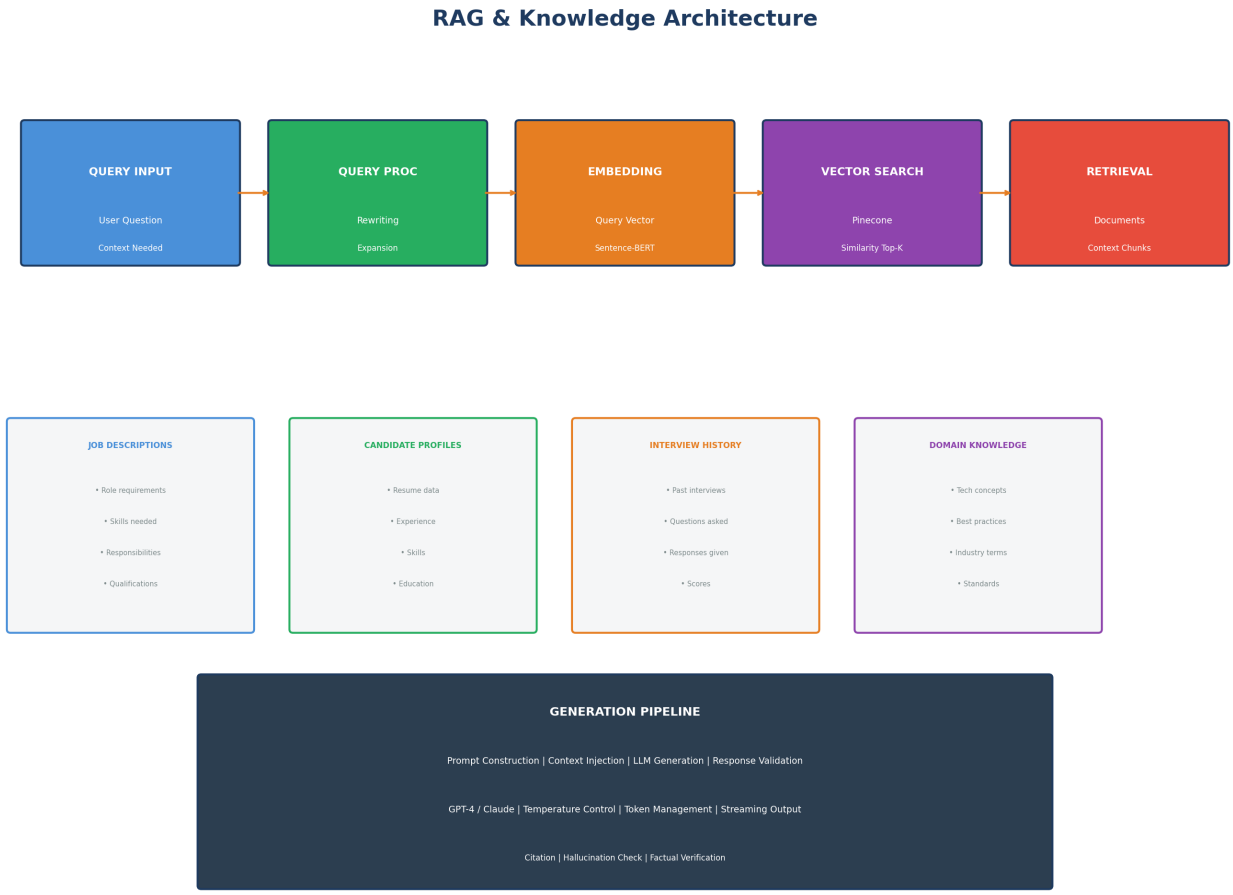

Figure 8: RAG & Knowledge Architecture

8.1 RAG Component Definitions

Query Processing: Transforms user queries for optimal retrieval. Implements query rewriting to expand or clarify ambiguous queries, query expansion to add relevant terms, and query routing to direct searches to appropriate knowledge bases.

Embedding Generation: Converts text into dense vector representations. Uses Sentence-BERT, E5, or OpenAI embeddings for semantic encoding. Supports multi-vector representations for complex queries.

Vector Search: Finds semantically similar documents using approximate nearest neighbor algorithms. Implements HNSW, IVF, or flat indexing for different latency-accuracy tradeoffs. Returns top-k most relevant chunks.

Re-ranking: Refines initial retrieval results using cross-encoder models. Scores query-document pairs more accurately than bi-encoder embeddings. Improves precision of retrieved context.

Context Injection: Integrates retrieved documents into the LLM prompt. Implements chunk selection, ordering, and formatting strategies. Handles context window limitations through intelligent truncation.

8.2 Knowledge Sources

Source	Content Type	Update Frequency	Use Case
Job Descriptions	Role requirements, skills, responsibilities	On-demand	Question generation, evaluation criteria
Candidate Profiles	Resumes, experience, education	Per interview	Personalized questions, profile matching
Interview History	Past questions, responses, scores	Real-time	Adaptive questioning, consistency checks
Domain Knowledge	Technical concepts, best practices	Monthly	Technical question validation, answer scoring
HR Guidelines	Company policies, evaluation rubrics	Quarterly	Bias prevention, standardized criteria

Table 8: Knowledge Source Definitions

9. AI Design Patterns

The AI architecture implements established design patterns that ensure maintainability, scalability, and reliability. These patterns represent proven solutions to common challenges in AI system development and have been adapted specifically for the recruitment interview context.

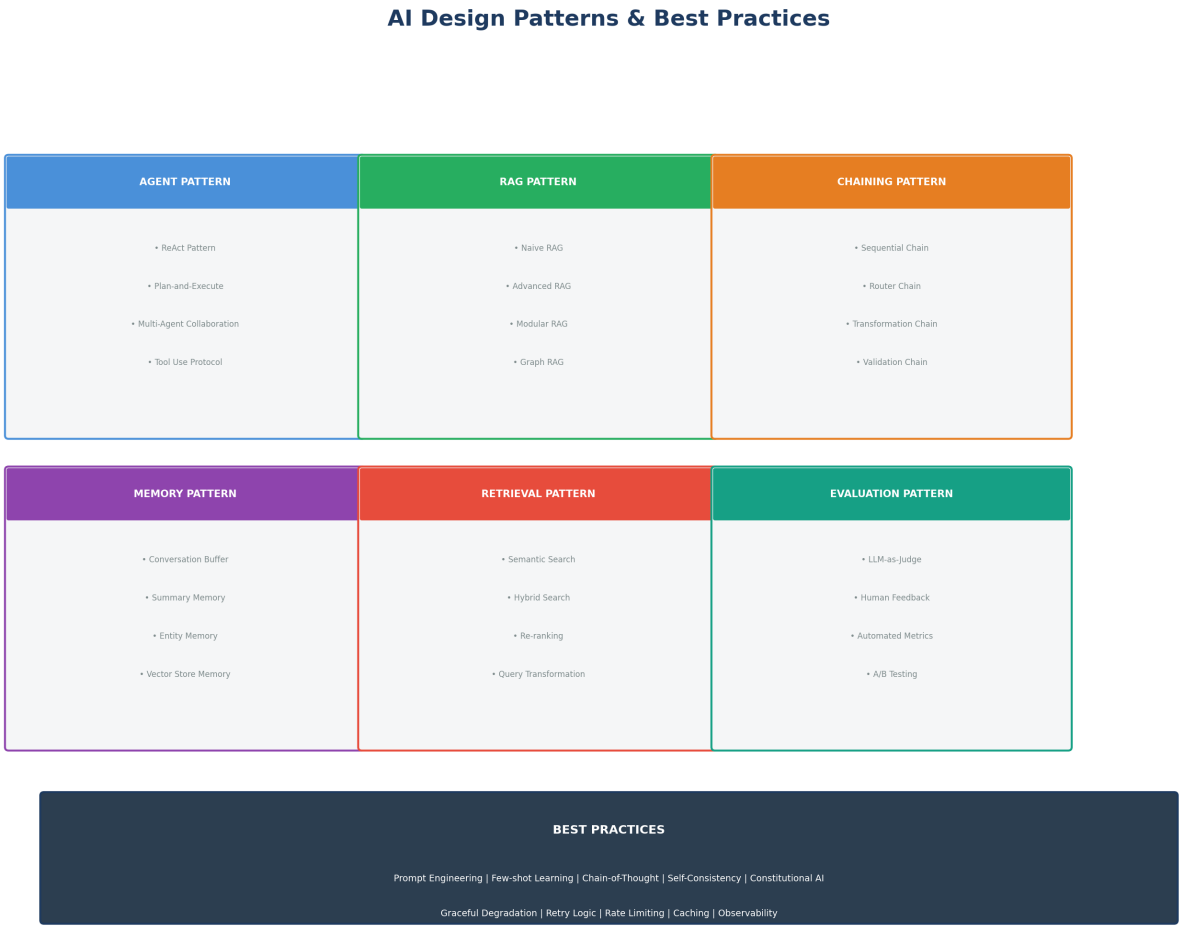

Figure 9: AI Design Patterns & Best Practices

9.1 Pattern Definitions

ReAct Pattern: Combines Reasoning and Acting in an interleaved manner. The agent reasons about what action to take, executes the action, observes the result, and reasons again. Enables dynamic decision-making based on intermediate results.

Plan-and-Execute: Separates planning from execution. First generates a complete plan of actions, then executes them sequentially. Useful for complex, multi-step tasks where upfront planning improves efficiency.

Multi-Agent Collaboration: Distributes tasks among specialized agents that work together. Each agent has specific expertise (NLP, Vision, Voice) and they communicate to solve complex problems collaboratively.

RAG Pattern: Augments LLM generation with retrieved context from a knowledge base. Reduces hallucinations and enables knowledge updates without model retraining. Implements dense retrieval for semantic matching.

Chain of Thought: Elicits reasoning by prompting the model to show its work. Improves performance on complex reasoning tasks by making the inference process explicit.

Constitutional AI: Implements self-critique and revision cycles. The model evaluates its outputs against defined principles and revises accordingly. Helps ensure outputs meet quality and safety standards.

9.2 Memory Patterns

- Conversation Buffer: Maintains a sliding window of recent messages. Simple but effective for short conversations. Memory usage grows linearly with conversation length.
- Summary Memory: Compresses conversation history into summaries. Enables longer conversations within context limits. May lose important details in summarization.
- Entity Memory: Extracts and tracks entities mentioned in conversation. Useful for maintaining facts about candidates throughout the interview. Supports structured queries.
- Vector Store Memory: Stores conversation turns as embeddings. Enables semantic retrieval of relevant past interactions. Scales to very long conversation histories.

10. Model Serving Architecture

The Model Serving architecture provides the infrastructure for deploying, managing, and scaling ML models in production. It implements best practices for inference optimization, resource utilization, and operational reliability.

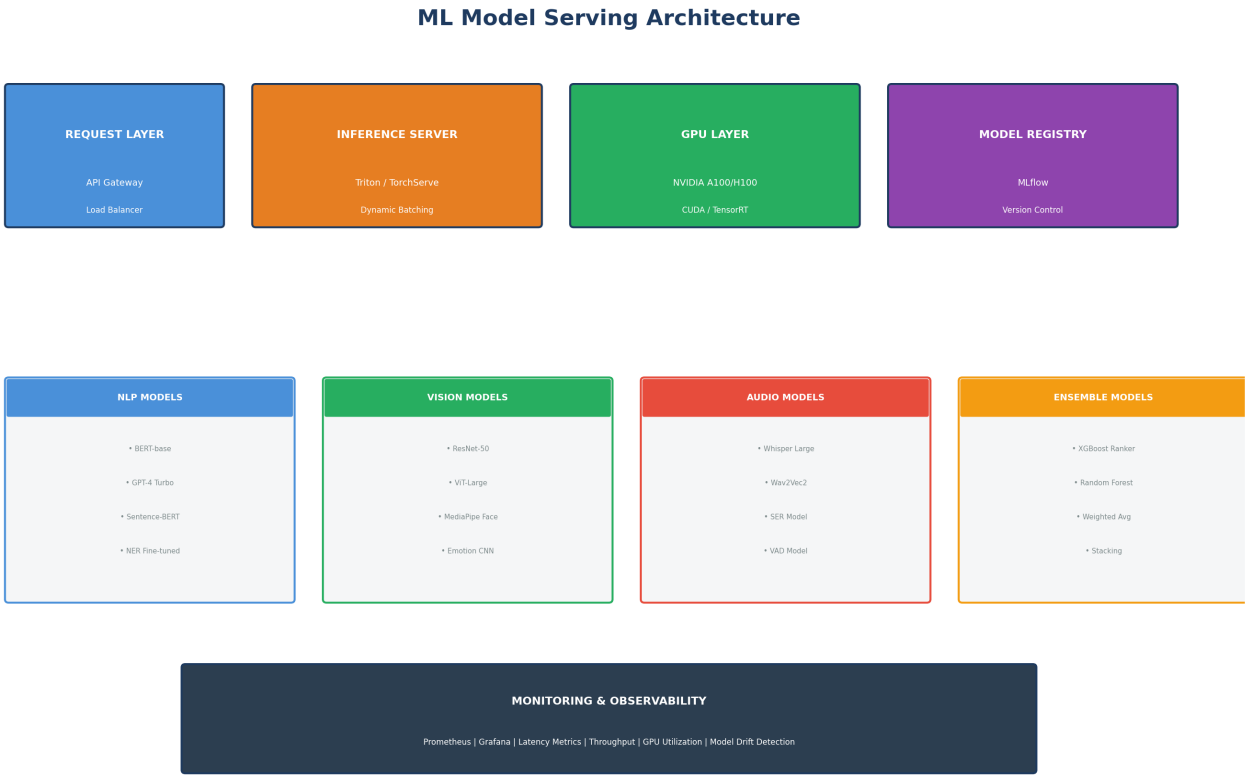

Figure 10: ML Model Serving Architecture

10.1 Serving Infrastructure

Inference Server: Triton Inference Server or TorchServe for model deployment. Supports multiple frameworks (PyTorch, TensorFlow, ONNX) with unified API. Implements dynamic batching for throughput optimization.

GPU Layer: NVIDIA A100/H100 GPUs for accelerated inference. CUDA and TensorRT for kernel optimization. Multi-instance GPU (MIG) for resource partitioning.

Model Registry: MLflow for model versioning and lifecycle management. Stores model artifacts, metrics, and metadata. Supports model promotion workflows and rollback.

Auto-scaling: Kubernetes Horizontal Pod Autoscaler for automatic scaling. Metrics-based scaling on GPU utilization, request queue depth, and latency targets.

10.2 Model Inventory

Category	Models	Framework	GPU Memory
NLP	BERT-base, GPT-4 Turbo, Sentence-BERT, NER Fine-tuned	PyTorch, ONNX	4-16 GB

Vision	ResNet-50, ViT-Large, MediaPipe Face, Emotion CNN	TensorFlow, ONNX	2-8 GB
Audio	Whisper Large-v3, Wav2Vec2, SER Model, VAD	PyTorch, ONNX	4-10 GB
Ensemble	XGBoost Ranker, Random Forest, Weighted Average	XGBoost, sklearn	CPU

Table 9: Model Inventory